Midterm Part 2: The Bus

Start Assignment

- Due Tuesday by 5pm
- Points 25
- Submitting a file upload
- Available after Oct 12 at 2pm

Midterm Part 2: The Bus

> Upload your code before you demonstrate.
>
> Your Part 2 Python goes to **this assignment**, and must be showing as submitted in Canvas **before** the TA starts your week 2 demonstration.
>
> Part 1 and Part 2 are graded independently, so this applies again even though you uploaded last week.

Your board has **two independent I²C buses**, and they are not loaded the same way. **25 points, demonstrated at the end of week 2.**

Bus	Devices on it	Test points	You have driven it in
Bus 0	temperature, humidity, pressure — three devices	SCL_0, SDA_0	Lab 5, and every lab since
Bus 1	capacitive sensor and the LSM303 — two devices	SCL_1, SDA_1	Part 1 of this midterm

More devices means more pins, more pads and more trace on the wire, which means more capacitance. In Lab 7 you measured a rise time on one bus and turned it into a capacitance. Now do both, and compare them.

You already have traffic on both

Bus 0 has your working ADT7420 read from Lab 5. Bus 1 has the LSM303 read you built in Part 1. No new firmware is needed here beyond pointing the PMOD marker output at whichever transaction you are capturing.

2a. Capture the same edge on both buses (10 points)

Use the four labelled through-holes: `SCL_0`, `SDA_0`, `SCL_1`, `SDA_1`.

- Trigger the oscilloscope from the PMOD marker you added in Lab 7, set to fire on the state just before the acknowledgment.
- On each bus, capture the SDA edge across that acknowledgment.
- Measure the **10–90% rise time** and the fall time, computed in Python from the captured array. No cursors.
- Use the PMOD ground pin adjacent to your signal pin as the scope reference.

> Trigger on the same event on both buses. Comparing two different edges and calling the difference capacitance is not a measurement.

2b. Do it many times (10 points)

One edge is an anecdote. Capture **at least 100** edges on each bus, automatically, and fit every one.

- Plot the two rise-time distributions as histograms on common axes.
- Report the mean and standard deviation for each bus.
- Do the same for the fall times.

2c. Turn the rise times into capacitance (5 points)

- For each bus, extract the RC time constant from your mean rise time and compute the bus capacitance. The pull-ups are R4 to R7 on the schematic, one per line.
- Report both capacitances and the difference between them.
- State the maximum SCL frequency each bus can actually support, using the rise-time limits in the I²C specification.

A number that exists nowhere else

The capacitance you extract appears on no datasheet, no schematic and no bill of materials. It is a property of your particular board that can only be obtained by measuring it. That is the whole point of the exercise.

Demo & Submission

Upload **one file** to this assignment, with exactly this name. Not zipped, not pasted into a document.

Filename	Contents
`midterm_p2_python.py`	the capture loop, the edge fitting, the histograms and the capacitance calculation

Your written work goes to the separate **Midterm – Report 2** assignment, due at the same moment. Code does not go there.

> Upload your code before you demonstrate.
>
> The TA checks Canvas before starting and will not begin until your files are showing as submitted.
>
> A demonstration with no code submitted scores **zero** for this part, whatever the bench did. If something is missing or misnamed when the TA looks, fix it there and then — caught at the bench it costs thirty seconds.

Due: First 60 minutes of the session at the end of week 2, the same deadline as Report 2. Late submission costs 10% per hour.

What the TA Checks, at the end of week 2

#	The TA does this	This must happen
0	**Opens this assignment in Canvas before starting**	**your Part 2 Python is submitted and correctly named.**
1	Looks at how the scope is triggered	from the PMOD marker, on the state just before the acknowledgment, and the *same* event on both buses
2	Looks at one captured edge per bus	rise and fall times computed in Python from the array, not read off cursors
3	Looks at your two histograms	at least 100 edges per bus, on common axes, with mean and standard deviation for each

4	Asks for the two capacitances	one per bus, with the pull-up value taken from R4 to R7 on the schematic
5	Asks for the maximum SCL each bus permits	a number per bus, from the I²C rise-time limits rather than the datasheet maximum